

Clips Bot – API usage walkthrough

Step-by-step script for the YouTube API Services audit · Channel: **Roty** (UCz0-X39y1B_BAO_yFCLMS8w) ·

Project contact: rotypro8@gmail.com

Screencast of the full flow (unlisted): <https://youtu.be/zlLOnuSHzQc>

What Clips Bot is

Clips Bot is a personal tool, used only by the channel owner, that prepares short vertical videos (Shorts) for the owner's own YouTube channel. The owner controls it from a private Telegram chat. Every video has to be approved there by the owner first. Nothing is uploaded automatically without approval. It uses the YouTube API Services to:

- **upload and schedule** approved videos on the owner's channel;
- **manage** those uploads (list them and cancel a scheduled publication);
- **track analytics** of the owner's own videos (views, average view duration, retention);
- **perform public searches** for public Shorts that mention the creators the channel works with, to find which moments became popular.

Access	Used for
OAuth 2.0 (installed app), scope <code>youtube.upload</code>	<code>videos.insert</code>
OAuth 2.0, scope <code>youtube</code>	<code>videos.update</code> (cancel a schedule), <code>channels.list</code> , <code>playlistItems.list</code>
OAuth 2.0, scope <code>yt-analytics.readonly</code>	YouTube Analytics API <code>reports.query</code> (own channel only)
API key	<code>search.list</code> and <code>videos.list</code> for public data (public search)

The interface language was switched to English for this demo with the bot command `/lang en`.

Step 1 — Approval in Telegram

The owner sends `/demo`. The bot creates a short test video (color bars, no third-party content) and sends it to the private chat with the title, description and hashtags it will use, and the buttons  **Upload to YouTube** /  **Don't upload**. Nothing reaches YouTube until the owner presses .

API: none in this step (approval happens in Telegram).

[Screenshot 1 — Telegram: the test video with Title / Description / Hashtags and the " Upload to YouTube" button]

Step 2 — Upload and schedule

The owner presses  **Upload to YouTube**. The bot uploads the file to the owner's channel as **private** with a scheduled publication time (tomorrow at 13:00, Argentina time). It replies with the video id and a direct link to YouTube Studio, and the button changes to  **Scheduled for 13:00** /  **Cancel**.

API: `videos.insert` (resumable upload, `part=snippet,status`) with `status.privacyStatus = "private"` and `status.publishAt = tomorrow 16:00 UTC`; snippet: title, description, tags, `categoryId`.

[Screenshot 2 — Telegram: "Uploaded with videos.insert ... scheduled (publishAt) for ...", with the video id and the Studio link]

Step 3 — End result in YouTube Studio

In YouTube Studio → Content, the uploaded video appears in the channel as **Private**, with the title, description and the publish time (publishAt) sent by the bot. Because the API project has not been audited yet, YouTube keeps videos uploaded through the API locked as private, so the scheduled publication cannot happen yet. That is expected, and it is why we are requesting this audit.

API: none (YouTube Studio shows the result of the videos.insert call).

[Screenshot 3 — YouTube Studio → Content: the test video, visibility “Private” (open its details to show the schedule)]

Step 4 — Manage: list and cancel the schedule

The owner sends `/subidas`: the bot lists every scheduled upload (channel, publish time, video id) with a **✗ Cancel** button. Pressing it removes the scheduled publication. The video is not deleted: it stays private, with no publish date. Studio shows the change.

API: `videos.update (part=status)` with `privacyStatus = "private"` and no `publishAt`, which removes the schedule. The list itself comes from the bot's own database of the uploads it made.

[Screenshot 4 — Telegram: `/subidas` listing the scheduled test video with "✗ Cancel #N"]

[Screenshot 5 — Telegram "Schedule #N cancelled with `videos.update`" + YouTube Studio: the video private, without a scheduled date]

Step 5 — Track analytics

The owner sends `/metrics refresh`. The bot requests the latest statistics of the owner's own Shorts and shows, per video: views, average % viewed, % of viewers still watching at the end, and average view duration. The same refresh runs automatically once a day. Every stored statistic is re-requested before 30 days. Stored data is deleted only if the video no longer exists on the channel, or if the owner revokes access.

```
API: channels.list (mine=true, part=contentDetails) → playlistItems.list (uploads playlist) to list the channel's videos; YouTube Analytics API reports.query with ids=channel==MINE, metrics views, averageViewDuration, averageViewPercentage (dimension video) and audienceWatchRatio (dimension elapsedVideoTimeRatio).
```

[Screenshot 6 — Telegram: “Your latest Shorts — YouTube Analytics” with views, % viewed, % at the end, duration]

Step 6 — Public search

The owner sends `/publicos <creator>` (for example `/publicos davooxeneize`). The bot searches YouTube for **public** Shorts that mention that creator, and replies right away with each Short's title, channel, view count and link. Only public data is used: title, channel name and view count. It is used only to find which moments of the creator became popular, and it is deleted after 30 days.

```
API (API key): search.list (q = creator names, type=video, videoDuration=short, order=viewCount, publishedAfter) → videos.list (part=snippet,statistics,contentDetails) for exact view counts and durations.
```

[Screenshot 7 — Telegram: “Public YouTube search (search.list + videos.list)” and the list of public Shorts with channel, views and link]

Data handling

- Only the owner's own channel is accessed with OAuth. Other channels: public data only, through the API key.
- Stored: video id, scheduled date and the statistics listed above. Statistics are refreshed before 30 days. They are deleted if the video no longer exists or access is revoked, and on request within 7 days (rotypro8@gmail.com).
- Privacy policy: <https://s-fitti848.github.io/privacidad.html> · Terms: <https://s-fitti848.github.io/terminos.html>